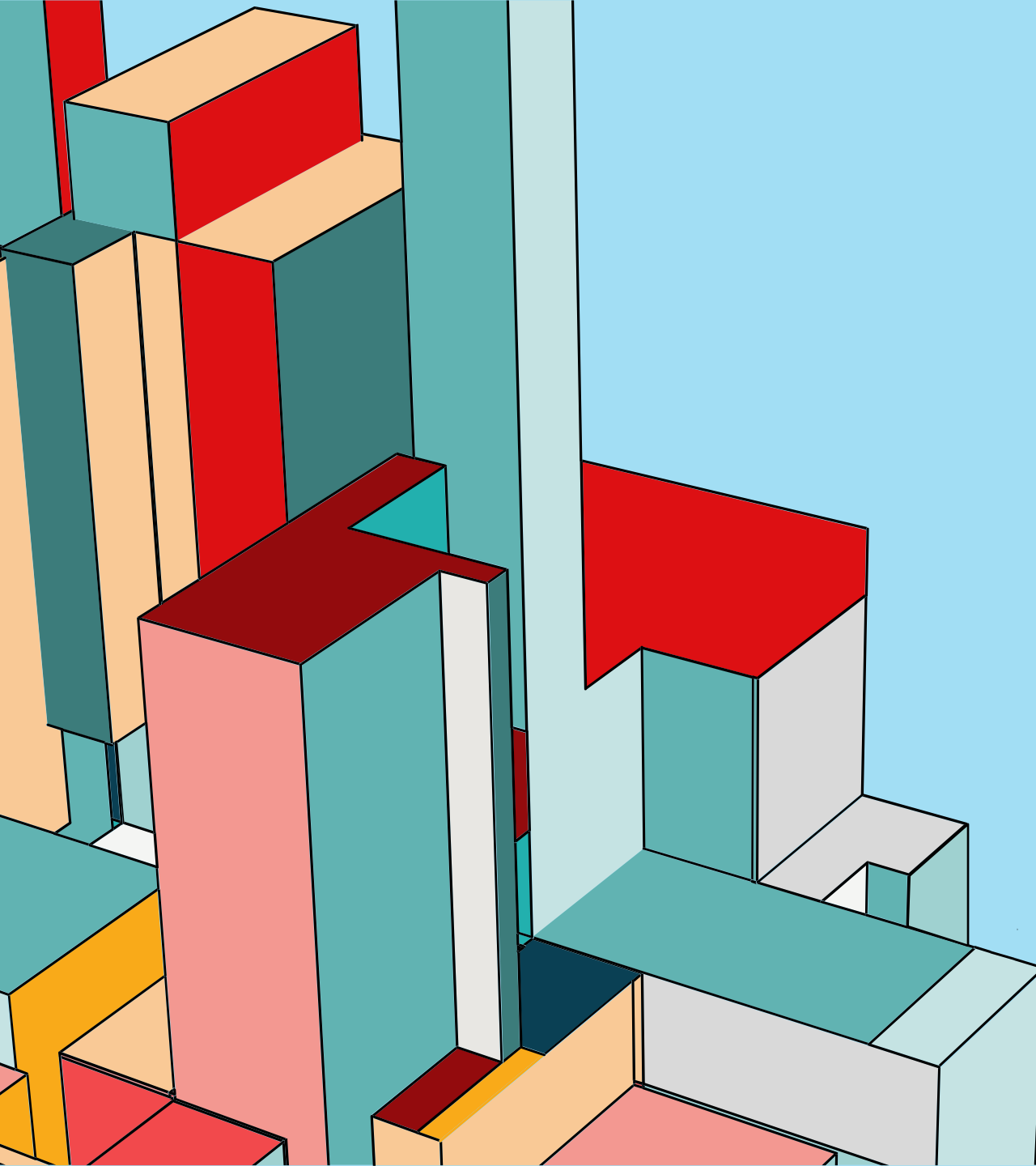


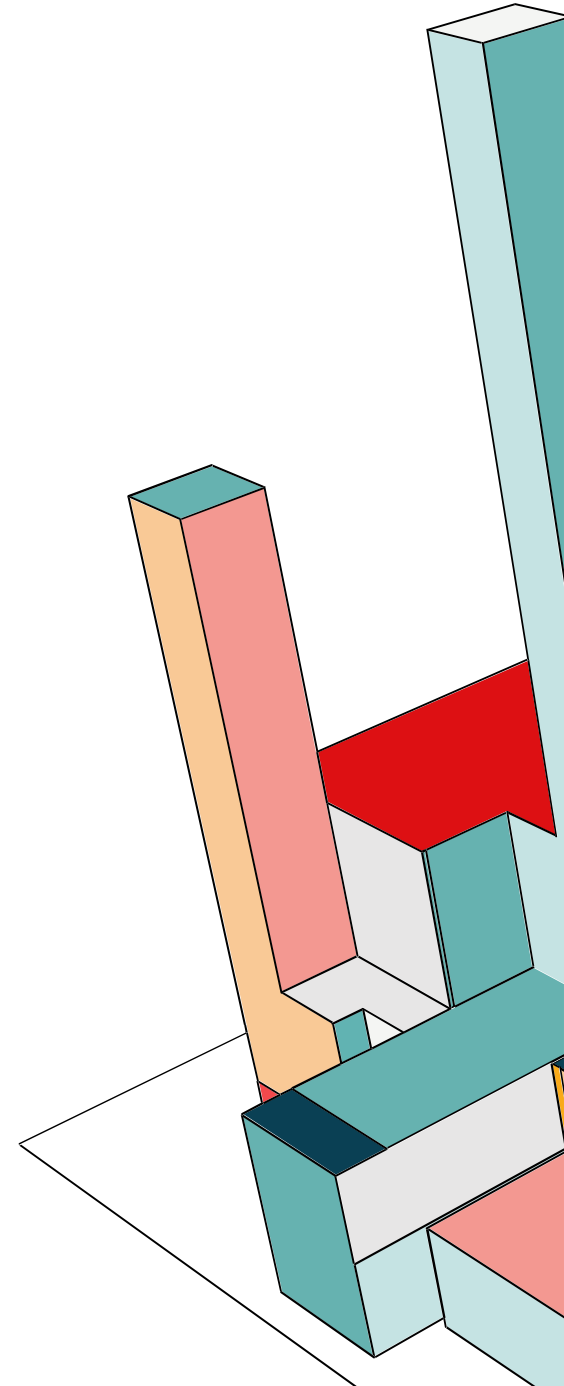


LECTURE 17 **DYNAMIC PROGRAMMING II**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS & REMINDERS

- HW 8 posted tonight; no lab this week



WHEN LAST WE MET...

Dynamic programming is an algorithm design process that takes full advantage of the space-for-time tradeoff.

It allows us to eliminate unnecessary recursion by recording the results of calculations whose answers we need to reuse.

To produce a dynamic programming algorithm:

1. Characterize the recursive substructure of the problem.
2. Write a recurrence relation that specifies the optimal value of a larger problem in terms of the optimal values of smaller problems.
3. Compute **and record** the values (and path to a solution) of every subproblem, in bottom-up fashion, until you reach the target size.
4. Reconstruct the optimal solution from the recorded information.

MAKING CHANGE

Recall the change-making problem: Given a collection D of denominations of coins $d_1 < d_2 < \dots < d_m$, make precisely n cents using the minimum number of coins. We assume $d_1 = 1$ so that all values are possible.

We saw that for the standard U.S. system with 1¢, 5¢, 10¢, and 25¢ coins, a greedy algorithm works just fine.

We also saw that greedy does not, in general, produce optimal solutions (for example, if we remove nickels from that list).

Do now: Create a dynamic programming algorithm to solve the general change-making problem. Write the pseudocode.

MAKING CHANGE

Step #1: Characterize the recursive substructure of the problem.

Say you select coin d_i . Now what?

Clearly, you now need to solve the problem for $n - d_i$.

Since we don't know which d_i to pick, we'll try all of them.

Step #2: Express that structure as a recurrence relation.

$$OPT(n) = 1 + \min_{i:n \geq d_i} F(n - d_i); OPT(0) = 0$$

MAKING CHANGE

$$OPT(n) = 1 + \min_{i:n \geq d_i} F(n - d_i); OPT(0) = 0$$

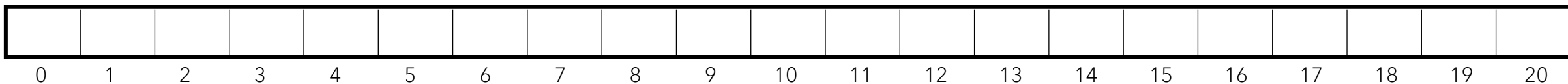
Step #3: Compute and record the values in order.

```
def change(d,n):  
    OPT[0] = 0  
    for val = 1 to n:  
        best = ∞; i = 1  
        while i <= m AND d[i] <= val:  
            best = min(best, OPT[val - d[i]])  
            i = i + 1  
        OPT[val] = 1 + best  
    return OPT[n]
```

MAKING CHANGE – EXECUTING THE ALGORITHM

Let $D = \{1,3,4,6,11\}$ and $n = 20$. Fill in the DP array (OPT).

```
def change(d,n):  
    OPT[0] = 0  
    for val = 1 to n:  
        best =  $\infty$ ; i = 0  
        while i < m AND d[i] <= val:  
            best = min(best, OPT[val - d[i]])  
            i = i + 1  
        OPT[val] = 1 + best  
    return OPT[n]
```



SUBMITTY POLL #1

What is $\text{OPT}(20)$?



MAKING CHANGE – EXECUTING THE ALGORITHM

Let $D = \{1,3,4,6,11\}$ and $n = 20$. Fill in the DP array (OPT).

```
def change(d,n):  
    OPT[0] = 0  
    for val = 1 to n:  
        best =  $\infty$ ; i = 0  
        while i < m AND d[i] <= val:  
            best = min(best, OPT[val - d[i]])  
            i = i + 1  
        OPT[val] = 1 + best  
    return OPT[n]
```

9

0	1	2	1	1	2	1	2	2	2	2	1	2	3	2	2	3	2	3	3	3
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

THE KNAPSACK PROBLEM

A classic problem in computer science is the *knapsack problem*: You have a container (the knapsack) capable of holding a maximum weight W . There is a collection of n discrete objects: each of them has a weight w_i and a value v_i .

The task is to select a subset of the objects that maximizes the total value without exceeding the knapsack's capacity.

There are actually several variants on this problem; this particular one is the *0-1 Knapsack Problem* since there's only one copy of each item available, and we can take it or leave it.

As long as the objects are indivisible (i.e. you can't take fractions of an object), all of the variants are NP-complete.

THE KNAPSACK PROBLEM

Inputs: $W, w_1 \dots w_n, v_1 \dots v_n$

Let's construct a dynamic programming algorithm for this problem. What does the recursive structure look like?

This is once again a "use it, or don't" problem (i.e. a selection problem). So let's think about item n :

- If we don't use it, our best solution is the best solution for which smaller problem?
- If we do use it, our best solution is the best solution for which smaller problem... and modified by what?

THE KNAPSACK PROBLEM

Inputs: $W, w_1 \dots w_n, v_1 \dots v_n$

Let's construct a dynamic programming algorithm for this problem. What does the recursive structure look like?

This is once again a "use it, or don't" problem (i.e. a selection problem). So let's think about item n :

- If we don't use it, our best solution is the same as best solution for the problem with items $1 \dots (n - 1)$.
- If we do use it, our best solution is item n plus the best solution for the problem with items $1 \dots (n - 1) \dots$
 - ... but we can't carry as much anymore!

THE KNAPSACK PROBLEM

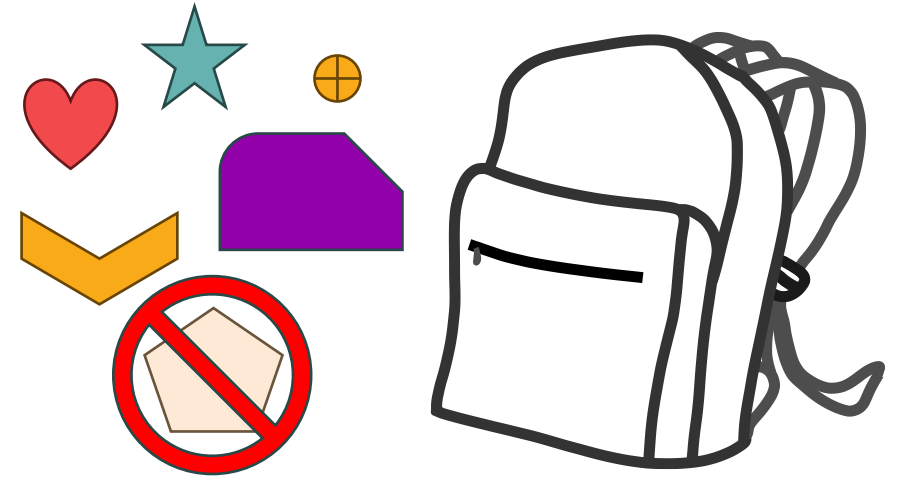
What should our recurrence relation be?

Key observation: The optimal selection of items depends on both the list of items and our remaining carrying capacity.

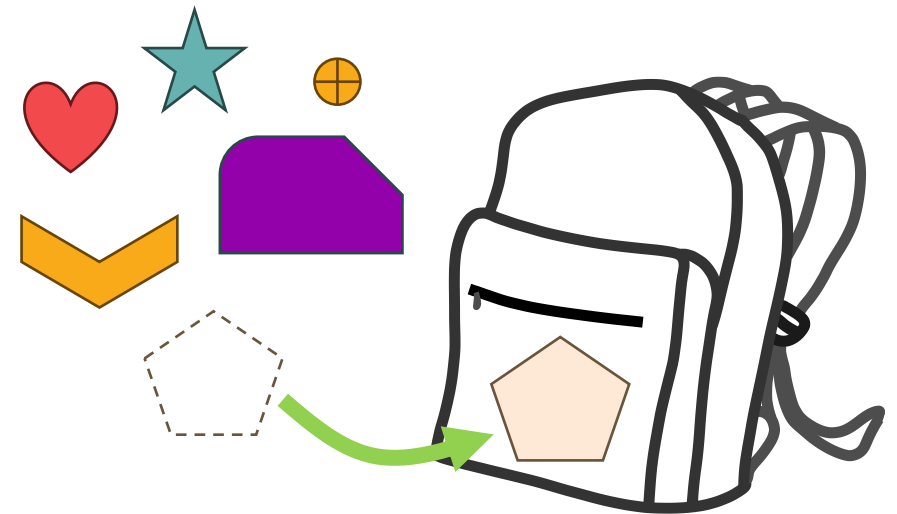
$$OPT(n, W) = \max($$

$$OPT(n - 1, W),$$

$$v_n + OPT(n - 1, W - w_n))$$



OR



THE KNAPSACK PROBLEM

We need to make one more adjustment and add base cases:

$$\begin{aligned} &OPT(n, W) \\ &= \begin{cases} OPT(n-1, W) & \text{if } w_n > W \\ \max(OPT(n-1, W), v_n + OPT(n-1, W - w_n)) & \text{otherwise} \end{cases} \end{aligned}$$

$$OPT(n, 0) = OPT(0, W) = 0 \text{ for all nonnegative } n, W$$

This accounts for cases where object n simply doesn't fit.

What kind of data structure will we need to record the answers to the calculations this time?

DYNAMIC PROGRAMMING WITH A 2-D ARRAY

We have two inputs, and making a selection can shrink the problem on one or both of these axes. So we must build a two-dimensional array to record answers to subproblems.

	0	1	2	...	W-2	W-1	W
0	0	0	0	0	0	0	0
1	0						
2	0						
...	0						
n-2	0						
n-1	0						
n	0						Target!

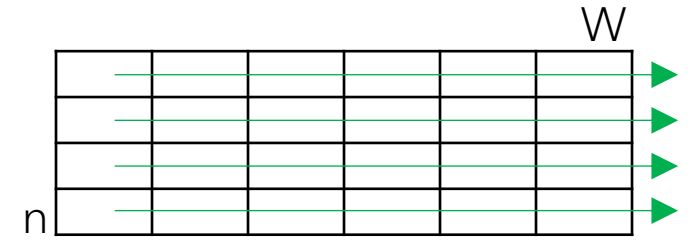
DYNAMIC PROGRAMMING WITH A 2-D ARRAY

	0	1	2	...	W-2	W-1	W
0	0	0	0	0	0	0	0
1	0						
2	0						
...	0						
n-2	0						
n-1	0						
n	0						Target!

With 2-D (or more) dynamic programming arrays, a potentially important question arises: In what order do we need to fill in the table?

For this particular problem, it doesn't matter too much – but we do need to decide in order to write the code. Let's go across first.

THE KNAPSACK PROBLEM: PSEUDOCODE



```
def knap(n,W,vals,wts):
```

```
    Declare OPT array: (n+1) by (W+1)
```

```
    Set  $OPT[i,0]$  and  $OPT[0,j] = 0$  for all valid  $i,j$ 
```

```
    for  $i = 1$  to  $n$ :
```

```
        for  $j = 1$  to  $W$ :
```

```
            if  $wts[i] > j$ :
```

```
                 $OPT[i,j] = OPT[i-1,j]$ 
```

```
            else:
```

```
                 $use\_it = vals[i] + OPT[i-1,j-wts[i]]$ 
```

```
                 $OPT[i,j] = \max(OPT[i-1,j], use\_it)$ 
```

```
    return  $OPT[n,W]$ 
```



Note the order of the loops – this was set by our decision to go across first.

What's our runtime?

ANALYSIS OF KNAPSACK DP

There are two nested loops – one executes n times, and one executes W times.

The interior of the loops is constant time. Thus, the overall runtime is $O(nW)$. The space complexity – i.e. the size of the array – is also $O(nW)$.



ANALYSIS OF KNAPSACK DP

We said up front that Knapsack is NP-complete. What gives?

Our runtime evaluation (and membership in P and/or NP) looks at how the runtime scales with the size of the input. And, at its heart, the size of the input is its length in bits.

For the item values and weights – lists of objects – doubling the number of bits simply doubles the number of objects. That is, the relationship between the input size and n is linear.

For the knapsack capacity – a value – doubling the number of bits squares the size of the value. The relationship between the input size and W is exponential.

PSEUDOPOLYNOMIAL ALGORITHMS

The runtime of our Knapsack DP algorithm is $O(nW)$ – but since W is an exponential function of the input size, we cannot accurately label this as a polynomial-time algorithm.

We call algorithms that have a polynomial dependence upon an input value *pseudopolynomial*. This term means a few things in practice:

1. When that input value is small, we can often handle quite large lists of inputs at reasonable speed.
2. OTOH, when that input value is large, even small input lists can be infeasible to solve precisely.
3. You don't get to claim the Millenium Prize for $P=NP$.

EXECUTING THE ALGORITHM

Let's try it out with a small example.

The items are shown in the table at right, and our capacity $W = 10$.

Create and fill in the dynamic programming table now.

Weights	2	3	4	5
Values	2	4	5	7

EXECUTING THE ALGORITHM & SHORT BREAK

Let's try it out with a small example.

The items are shown in the table at right, and our capacity $W = 10$.

Weights	2	3	4	5
Values	2	4	5	7

Create and fill in the dynamic programming table now.

We'll take 10 minutes now for our break + time to work on this problem.

EXECUTING THE ALGORITHM

Let's try it out with a small example.

The items are shown in the table at right, and our capacity $W = 10$.

Weights	2	3	4	5
Values	2	4	5	7

Create and fill in the dynamic programming table now.

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	2	2	2	2	2	2	2	2	2
2	0	0	2	4	4	6	6	6	6	6	6
3	0	0	2	4	5	6	7	9	9	11	11
4	0	0	2	4	5	7	7	9	11	12	13

Weights	2	3	4	5
Values	2	4	5	7

RECONSTRUCTING THE SOLUTION

	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	2	2	2	2	2	2	2	2	2
2	0	0	2	4	4	6	6	6	6	6	6
3	0	0	2	4	5	6	7	9	9	11	11
4	0	0	2	4	5	7	7	9	11	12	13

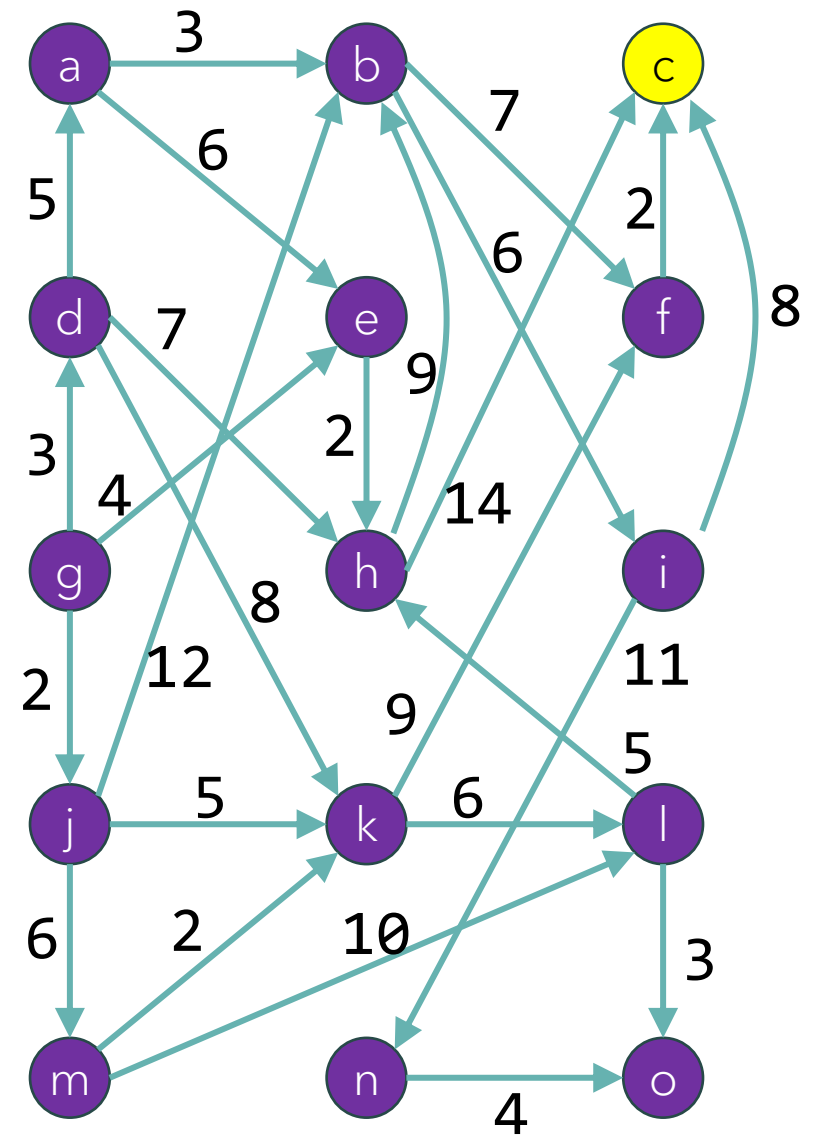
We now know the optimal set has value 13. But what set is it?

We can work backwards from the final answer...

- $\text{OPT}(4,10) = 13$ and $\text{OPT}(3,10) = 11$. Since these are different, we must have used item #4. $10 - 5$ (the weight of item #4) is 5, so check...
- $\text{OPT}(3,5) = 6$ and $\text{OPT}(2,5) = 6$. We didn't use item #3.
- $\text{OPT}(2,5) = 6$ and $\text{OPT}(1,5) = 2$, so we must have used item #2...

DAGS & SHORTEST PATH

Recall that we can use dynamic programming to quickly find the minimum-weight path in any directed acyclic graph by using topological sort.

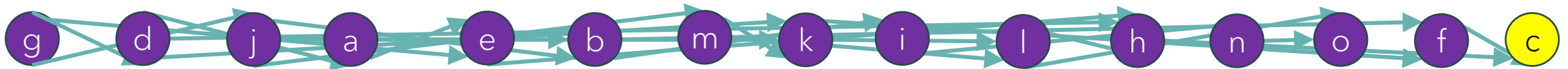


DAGS & SHORTEST PATH

Recall that we can use dynamic programming to quickly find the minimum-weight path in any directed acyclic graph by using topological sort.

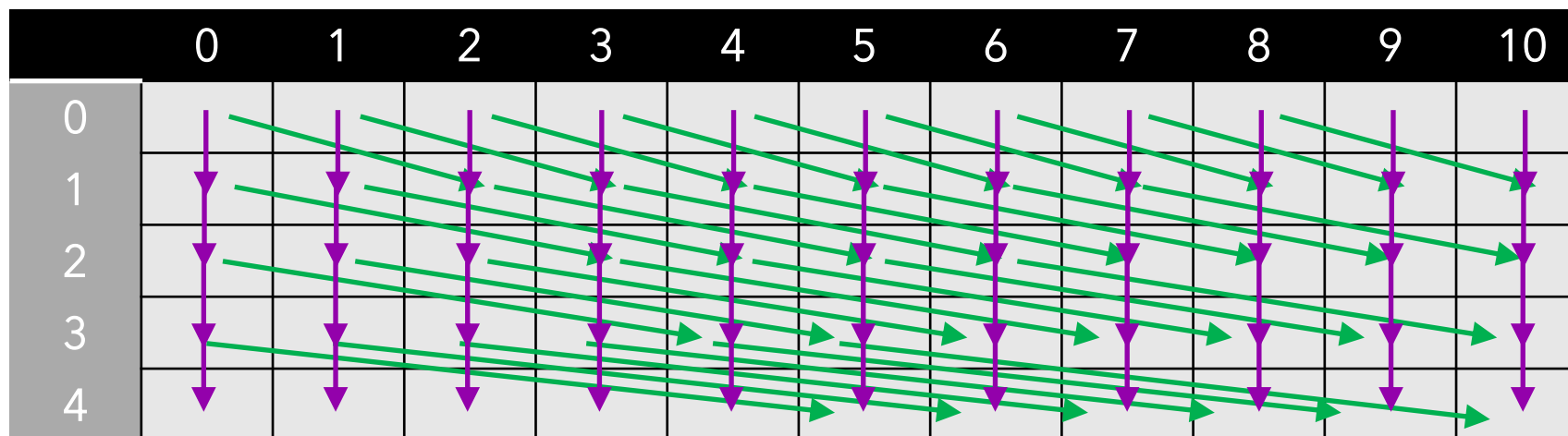
We could, of course, find a maximum-weight path in exactly the same way.

The surprising thing is this: EVERY dynamic programming algorithm amounts to one of these two tasks.



Weights	2	3	4	5
Values	2	4	5	7

THE UNDERLYING DAG



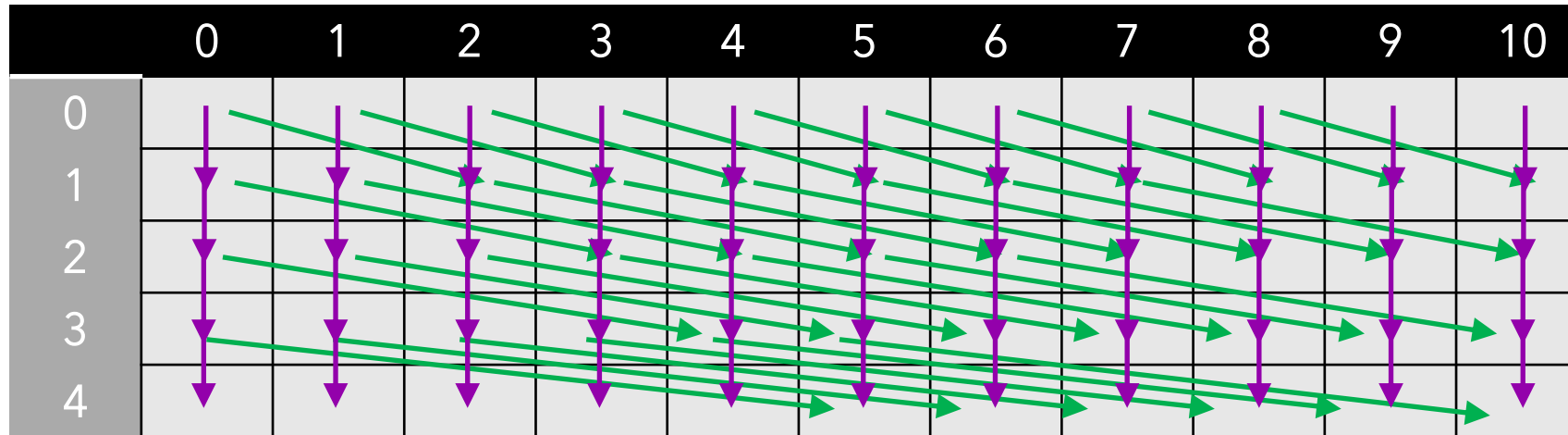
Consider each subproblem as a node, and each dependency as an edge, with the weight on the edge being the cost/utility of using that solution.

In our previous example, we want to maximize weights. $OPT(n, W)$ depends on $OPT(n - 1, W)$; these edges would have weight 0.

$OPT(n, W)$ also depends upon $OPT(n - 1, W - w_n)$ (as long as that exists); these edges would have weight v_n .

Weights	2	3	4	5
Values	2	4	5	7

THE UNDERLYING DAG



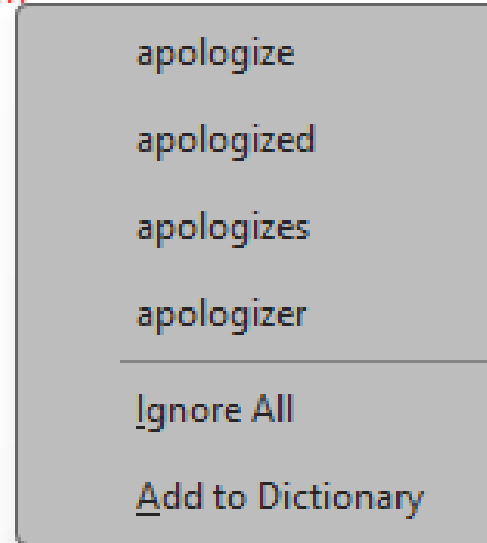
The heart of dynamic programming is this simple property:

There is some ordering $S_1, S_2, \dots$ of the subproblems such that the solution to S_k depends only upon some subset of the subproblems $S_1, \dots, S_{k-1}$.

Thus, if we solve the subproblems in this order, we need to solve each one only once.

STRING ALIGNMENT AND EDIT DISTANCE

Let me apolojize for



Spell checkers are a ubiquitous word processing tool nowadays.

How does the spell checker know what makes for a good suggestion?

Obviously, we would like words that are close to the misspelled word. But how do we quantify that notion of “closeness”?

STRING ALIGNMENT AND EDIT DISTANCE

One obvious approach is to see how many letters need to be changed to other letters.

apolojize
apologize

Obviously only 1 letter difference

sowbank
snowbank

But the technique doesn't work here! It looks like we need to make LOTS of changes.

We need to account for cases in which the misspelled word is shorter or longer than the "target" word.

STRING ALIGNMENT AND EDIT DISTANCE

We define an *edit* as one of three operations: an insertion of a single character, a deletion of a single character, or a substitution of one character for another.

These notions have been around for a long time – they were likely intuitively known to typesetters for centuries – but the rise of DNA/RNA sequencing and other bioinformatics drove a lot of study in this area.

We can then ask how many edits are needed to turn one word into another.

For example: SNOWY and SUNNY. We might use five edits as follows:

–	S	N	O	W	–	Y
S	U	N	–	–	N	Y

STRING ALIGNMENT AND EDIT DISTANCE

Now, you might be thinking that this could be done in fewer edits.

What is the minimum number of edits needed to turn SNOWY into SUNNY?

Answer now in the Submittity poll.



STRING ALIGNMENT AND EDIT DISTANCE

The *edit distance* (or *Levenshtein distance*) between two strings is the minimum number of insertions, deletions, and substitutions needed to turn one string into the other.

Technically, there are many different measures that are edit distances (e.g. Hamming distance) – they rely on different types of edits being allowed or not. But in common usage, when someone says “edit distance” they mean this particular metric.

Note that this meets the mathematical definition of a *distance metric*: It’s 0 only when the inputs are identical and positive otherwise, it’s symmetric, and it obeys the Triangle Inequality.

How might we calculate edit distance?

BEFORE YOU LEAVE

Please complete this feedback form about today's class.

